

IEC 62304: Brief Overview & Lean Firmware Team Implementation Guide

1. Introduction to IEC 62304 IEC 62304 is an international standard defining the life cycle requirements for the development of medical device software. It ensures safety, reliability, and compliance by structuring development, risk management, maintenance, and problem resolution processes.

2. Software Safety Classes IEC 62304 categorizes software into three classes based on risk:

- **Class A:** No injury or damage to health is possible.
- **Class B:** Non-serious injury is possible.
- **Class C:** Death or serious injury is possible.

3. SDLC Phases Defined in IEC 62304 Each phase includes specific required activities:

A. Software Development Planning

- Define lifecycle model
- Identify roles, responsibilities, tools, and interfaces

B. Software Requirements Analysis

- Document functional and safety-related requirements
- Trace requirements to hazards and risk controls

C. Software Architecture and Design

- Define high-level and detailed design
- Show data/control flow and interfaces

D. Software Implementation and Verification

- Code implementation with coding standards
- Unit and integration testing based on requirements

E. Software Integration and System Testing

- Integrate all components
- Validate against requirements
- Conduct risk-based testing

F. Software Release

- Define acceptance criteria
- Perform final verification
- Release documentation and baseline version

G. Maintenance Process

- Document change requests and perform impact analysis
- Re-verify modified components

H. Risk Management Integration

- Align with ISO 14971
- Identify, control, and verify software-related risks at all stages

I. Problem Resolution Process

- Record, investigate, and resolve anomalies
- Document corrective actions and verification

4. Lean Firmware Team Implementation Plan

For small teams, focus on **agile but traceable** methods:

- **Use lightweight tools:** Git + Markdown + Doxygen for code/doc traceability
- **Define minimal viable documentation:** Plan, requirements, design, risk matrix, test reports
- **Automate testing:** Use CI/CD for unit testing and static analysis
- **Assign cross-functional roles:** One person may handle multiple roles with segregation ensured through peer reviews
- **Iterative risk analysis:** Update risk control documentation during each sprint
- **Traceability matrix:** Use Excel or simple tools to trace requirements to design, implementation, and tests

5. Recommended Tools

- Project Tracking: Jira, Trello, or GitHub Projects
- Documentation: Sphinx, MkDocs, or LaTeX
- Version Control: Git
- Testing: Unity (unit testing), CMock, or Google Test
- Static Analysis: PC-lint, SonarQube